

**МИНОБРНАУКИ РОССИИ**  
**САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ**  
**ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ**  
**«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)**  
**Кафедра математического обеспечения и применения ЭВМ**

**ОТЧЕТ**  
**по лабораторной работе №2**  
**по дисциплине «Построение и анализ алгоритмов»**  
**Тема: Задача коммивояжера**

Студент гр. 4345

Гаврилов Е.В.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

## **Цель работы**

Изучить и применить на практике алгоритмы решения задачи коммивояжера.

## **Задание**

Напишите программу, решающую задачу коммивояжера. Нужно найти кратчайший маршрут, который проходит через все заданные города ровно один раз и возвращается в исходный город. Не все города могут быть напрямую связаны друг с другом.

## **Входные данные:**

**$n$  – количество городов ( $5 \leq n \leq 15$ ).**

Матрица расстояний между городами размером  $n \times n$ , где  $graph[i][j]$  обозначает расстояние от города  $i$  до города  $j$ . Если  $graph[i][j]=0$  (и  $i \neq j$ ), это означает, что прямого пути между городами нет.

## **Выходные данные:**

Минимальная стоимость маршрута, проходящего через все города и возвращающегося в начальный город.

Оптимальный путь в виде последовательности посещаемых городов, начинающейся и заканчивающейся в городе 0.

Если такого пути не существует, вывести "no path".

**Вариант 8.** Точный метод: динамическое программирование (не МВиГ), итеративная реализация. Приближённый алгоритм: АЛШ-2.

## Выполнение работы

В ходе работы были реализованы два алгоритма решения задачи коммивояжёра (TSP): приближённый эвристический алгоритм ближайшей вставки (класс `Alsh2`) и точный алгоритм динамического программирования Хелда-Карпа (класс `HeldKarp`). Оба алгоритма работают с графом, представленным матрицей расстояний между городами (класс `Graph`), и возвращают стоимость найденного маршрута и сам маршрут в виде последовательности городов, начинающейся и заканчивающейся в городе 0.

### Описание класса `Graph`

Класс хранит квадратную матрицу расстояний `matrix` и её размер `size` (количество городов). Метод `get_dist(u, v)` возвращает расстояние между городами `u` и `v`: если значение `matrix[u][v]` больше нуля либо `u` равно `v` (переход из города в самого себя), возвращается само значение `matrix[u][v]`; в противном случае (нулевое значение вне диагонали трактуется как отсутствие ребра между городами) возвращается `float("inf")`, означающая недостижимость.

#### *Задание 1*

В качестве приближенного решения реализован алгоритм включения ближайшего города ALSH-2. На каждом шаге он выбирает среди ещё не посещённых городов тот, который находится ближе всего к уже построенному маршруту, и вставляет его в то место маршрута, где это даёт наименьшее увеличение его стоимости.

### Описание функции `solve()` -> `Tuple[Union[int, str], Optional[List[int]]]`

Маршрут `path` инициализируется списком `[0, 0]` (выход из города 0 и немедленное возвращение в него), список `unvisited` заполняется номерами всех городов, кроме нулевого, а стоимость `cost` обнуляется.

Далее, пока список `unvisited` не пуст, выполняется следующий цикл:

- в переменную `best` записывается `None`, что означает отсутствие найденного на текущем шаге кандидата на вставку;
- для каждого непосещённого города `city` вычисляется величина `nearest` – минимальное расстояние от `city` до какого-либо из городов, уже включённых в маршрут (перебираются все города `path[:-1]`, то есть маршрут без учёта замыкающего повторного нуля);

– далее для каждой возможной позиции вставки position (то есть для каждого ребра (left, right) текущего маршрута, где  $\text{left} = \text{path}[\text{position}]$ ,  $\text{right} = \text{path}[\text{position} + 1]$ ) вычисляются расстояния  $\text{left\_dist} = \text{dist}(\text{left}, \text{city})$  и  $\text{right\_dist} = \text{dist}(\text{city}, \text{right})$ ; если хотя бы одно из этих расстояний равно бесконечности (соответствующего ребра не существует), данная позиция для данного города пропускается;

– иначе вычисляется величина increase – насколько увеличится стоимость маршрута при вставке города city между left и right:  $\text{increase} = \text{left\_dist} + \text{right\_dist} - \text{dist}(\text{left}, \text{right})$ ;

– формируется кандидат candidate в виде кортежа (nearest, increase, city, position); если best ещё не задан либо candidate меньше текущего best (кортежи сравниваются лексикографически: в первую очередь по близости города к маршруту, затем по приросту стоимости, затем по номеру города, затем по позиции), значение best заменяется на candidate;

– если по окончании перебора всех городов и позиций best остался равным None (ни один непосещённый город не удалось вставить ни в одну позицию – так происходит, если для оставшихся городов не существует подходящих рёбер), функция завершает работу и возвращает пару ("no path", None);

– иначе выбранный город city вставляется в маршрут на позицию position + 1 (функцией path.insert), удаляется из списка unvisited, а величина increase прибавляется к общей стоимости cost.

После того как список unvisited опустеет (все города включены в маршрут), функция возвращает пару (cost, path) – итоговую стоимость приближённого маршрута и сам маршрут.

### **Описание функции main()**

Из стандартного ввода считываются: количество городов n и n строк матрицы расстояний matrix. На основе матрицы создаётся экземпляр класса Graph, для которого запускается решатель Alsh2. Метод solve() возвращает найденную стоимость cost и маршрут path. Если path равен None (маршрут не найден), в стандартный вывод печатается строка "no path"; иначе построчно выводятся стоимость cost и сам маршрут path (номера городов через пробел).

## Задание 2

В качестве точного алгоритма решения задачи Коммивояжера реализован динамический алгоритм Хелда-Карпа.

**Описание функции solve() -> Tuple[Union[int, str], Optional[List[int]]]**

Пусть  $n$  – число городов,  $full\_mask$  – битовая маска ( $1 \leq full\_mask < 2^n$ ), в которой установлены все  $n$  битов (соответствует состоянию, когда посещены все города). Заводятся две таблицы размера  $(2^n) \times n$ :  $memo$ , хранящая для каждой пары (маска посещённых городов  $mask$ , текущий город  $u$ ) минимальную стоимость завершения маршрута из города  $u$  при условии, что уже посещены в точности города из  $mask$ , и  $parent$ , хранящая для той же пары оптимальный следующий город, в который нужно перейти из  $u$ .

Базовый случай: для маски  $full\_mask$  (когда все города уже посещены) для каждого города  $u$  значение  $memo[full\_mask][u]$  инициализируется расстоянием  $dist(u, 0)$  – стоимостью возврата из города  $u$  непосредственно в стартовый город  $0$ .

Далее выполняется заполнение таблицы  $memo$  для масок  $mask$  от  $full\_mask - 1$  до  $1$  в порядке убывания. Для каждой маски, не содержащей бит города  $0$  ( $mask \& 1$  равен нулю), обработка пропускается, поскольку любой маршрут обязан начинаться с посещённого города  $0$ . Для остальных масок перебираются все города  $u$ , входящие в  $mask$  (то есть уже посещённые), и для каждого такого  $u$  перебираются все города  $v$ , не входящие в  $mask$  (то есть ещё не посещённые): вычисляется расстояние  $dist = dist(u, v)$ , маска  $next\_mask = mask \mid (1 \ll v)$  (состояние после посещения города  $v$ ), и суммарная стоимость  $new\_dist = dist + memo[next\_mask][v]$ ; если  $new\_dist$  меньше текущего значения  $memo[mask][u]$ , значение  $memo[mask][u]$  заменяется на  $new\_dist$ , а  $parent[mask][u]$  запоминает город  $v$  как оптимальное следующее направление движения.

По завершении заполнения таблицы минимальная стоимость полного маршрута равна  $memo[1][0]$  (маска  $1$  означает, что посещён только стартовый город  $0$ , а текущий город – тоже  $0$ , то есть маршрут ещё не начат). Если это значение равно  $float("inf")$ , оптимального маршрута, посещающего все города и возвращающегося в город  $0$ , не существует, и функция возвращает пару ("no path", None).

Иначе выполняется восстановление самого маршрута: начиная с  $mask = 1$  и  $city = 0$ , на каждом шаге из таблицы  $parent$  извлекается  $next\_city = parent[mask][city]$ , вычисляется  $next\_mask = mask | (1 \ll next\_city)$ , город  $next\_city$  добавляется в список  $path$ , после чего  $city$  и  $mask$  обновляются значениями  $next\_city$  и  $next\_mask$  соответственно. Цикл повторяется, пока  $mask$  не станет равным  $full\_mask$  (все города посещены), после чего в конец маршрута добавляется возврат в город 0. Функция возвращает пару  $(min\_cost, path)$  – точную минимальную стоимость обхода и сам оптимальный маршрут.

### **Описание функции main()**

Считывание входных данных и вывод результата выполняются так же, как и в задании 1.

### **Сложность алгоритма**

Для алгоритма АЛШ-2, где  $n$  — число городов:

**по времени:** главный цикл выполняется  $n - 1$  раз (по одному разу на вставку каждого из оставшихся городов). На каждой итерации для каждого из оставшихся непосещённых городов (до  $n$  штук) вычисляется величина  $nearest$  и перебираются все позиции вставки в маршруте – выполняется работа порядка длины текущего маршрута, которая растёт от 1 до  $n$ . Суммарно по всем итерациям это даёт оценку  $O(n * \sum k) = O(n^3)$ ;

**по памяти:** основной объём занимает входная матрица расстояний размера  $n$  на  $n$ , то есть  $O(n^2)$ ; дополнительные структуры (маршрут  $path$ , список  $unvisited$ ) требуют  $O(n)$  памяти, поэтому итоговая оценка –  $O(n^2)$ .

Для алгоритма Хелда-Карпа, где  $n$  — число городов:

**по времени:**  $O(n^2 \cdot 2^n)$ , поскольку для каждой из  $2^n$  масок посещённых городов перебираются все пары городов  $(u, v)$ , что даёт  $O(n^2)$  операций на маску;

**по памяти:**  $O(n \cdot 2^n)$ , поскольку таблицы  $memo$  и  $parent$  имеют размер  $(2^n) \times n$  каждая.

Исходный код программы см. в приложении А.

Результаты тестирования алгоритма Хелда-Карпа см. в Таблице 1.

Результаты тестирования алгоритма включения ближайшего города см. в Таблице 2.

Таблица 1 – Результаты тестирования алгоритма Хелда-Карпа

| № | Ввод                                                                               | Вывод             | Комментарий                                       |
|---|------------------------------------------------------------------------------------|-------------------|---------------------------------------------------|
| 1 | 4<br>0 10 15 20<br>10 0 35 25<br>15 35 0 30<br>20 25 30 0                          | 80<br>0 1 3 2 0   | Найден оптимальный маршрут стоимостью 80          |
| 2 | 3<br>0 5 0<br>5 0 0<br>0 0 0                                                       | no path           | Вершина 2 изолирована, поэтому пути не существует |
|   | 5<br>0 1 13 23 7<br>12 0 15 18 28<br>21 29 0 33 28<br>23 19 34 0 38<br>5 40 7 39 0 | 78<br>0 4 2 3 1 0 | Найден оптимальный маршрут стоимостью 78          |

Таблица 2 – Результаты тестирования АВБГ

| № | Ввод                                                                               | Вывод             | Комментарий                                                                            |
|---|------------------------------------------------------------------------------------|-------------------|----------------------------------------------------------------------------------------|
| 1 | 4<br>0 10 15 20<br>10 0 35 25<br>15 35 0 30<br>20 25 30 0                          | 80<br>0 2 3 1 0   | Найден маршрут стоимостью 80, который не обязан был быть оптимальным, но так случилось |
| 2 | 3<br>0 5 0<br>5 0 0<br>0 0 0                                                       | no path           | Вершина 2 изолирована, поэтому пути не существует                                      |
| 3 | 5<br>0 1 13 23 7<br>12 0 15 18 28<br>21 29 0 33 28<br>23 19 34 0 38<br>5 40 7 39 0 | 86<br>0 1 3 2 4 0 | Найдено приближение, довольно точное (отклонение около 10%)                            |

## **Вывод**

Изучен и применен для поиска решения задачи Коммивояжера точный алгоритм динамического программирования – алгоритм Хелда-Карпа, и приближенный алгоритм для нахождения приближения решения этой же задачи – алгоритм включения ближайшего города.



## ПРИЛОЖЕНИЕ А

### ИСХОДНЫЙ КОД ПРОГРАММЫ

Файл TSP.py

```
#!/usr/bin/env python3

import sys

class Graph:
    def __init__(self, matrix):
        self.matrix = matrix
        self.size = len(matrix)

    def get_dist(self, u, v):
        dist = self.matrix[u][v]
        return dist if dist > 0 or u == v else float("inf")

class Alsh2:
    def __init__(self, graph, debug=False):
        self.graph = graph
        self.debug = debug

    def _log(self, message):
        if self.debug:
            print(message)

    def solve(self):
        n = self.graph.size
        path = [0, 0] # initial path
        unvisited = list(range(1, n))
        cost = 0 # 0 -> 0 takes 0 cost

        self._log("\n" + "=" * 20)
        self._log("ALSH-2 ALGORITHM")
        self._log(f"Number of cities: {n}")
        self._log(f"Initial path: {path}")
        self._log("=" * 20)

        while unvisited: # while there are cities to visit
            best = None # we want to find best city and its insertion
            position

            self._log(f"\nCurrent path: {path}")
            self._log(f"Unvisited cities: {unvisited}")

            for city in unvisited: # for each city not in path
                # find the smallest distance from the city to the current
                path

                nearest = min(
                    self.graph.get_dist(current, city)
                    for current in path[:-1]
                )

                # try to insert the city between each pair of
            neighbours
```

```

        for position in range(len(path) - 1):
            left = path[position]
            right = path[position + 1]
            left_dist = self.graph.get_dist(left, city)
            right_dist = self.graph.get_dist(city, right)

            # both new edges must exist
            if left_dist == float("inf") or right_dist ==
float("inf"):
                continue

            # calculate the increase in the path cost
            increase = (
                left_dist
                + right_dist
                - self.graph.get_dist(left, right)
            )
            candidate = (nearest, increase, city, position)

            self._log(
                f"\tTry city {city} between {left} and {right}:"
                f"nearest={nearest}, increase={increase}"
            )

            if best is None or candidate < best: # store the
best insertion
                best = candidate

        # no remaining city can be inserted into the current path
        if best is None:
            self._log("THERE IS NO PATH!")
            self._log("=" * 20 + "\n")
            return "no path", None

        _, increase, city, position = best
        path.insert(position + 1, city) # insert the nearest city
        unvisited.remove(city) # mark the city as visited
        cost += increase # update the path cost

        self._log(
            f"\tINSERT city {city} at position {position + 1}: "
            f"increase={increase}"
        )

        self._log(f"\nApproximate cost: {cost}")
        self._log(f"Approximate path: {' -> '.join(map(str, path))}")
        self._log("=" * 20 + "\n")
        return cost, path

class HeldKarp:
    def __init__(self, graph, debug=False):
        self.graph = graph
        self.debug = debug

    def _log(self, message):
        if self.debug:

```

```

        print(message)

    def _mask(self, mask):
        return format(mask, f"0{self.graph.size}b")

    def _visited(self, mask):
        return [city for city in range(self.graph.size) if mask & (1 <<
city)]

    def _print_memo(self, memo):
        if not self.debug:
            return

        n = self.graph.size
        self._log("=" * 20)
        self._log("\tDP table")
        self._log("mask\t" + "\t".join(f"u={u}" for u in range(n)))

        for mask, row in enumerate(memo):
            if (mask & 1):
                values = ["inf" if value == float("inf") else str(value)
for value in row]
                self._log(
                    f"{self._mask(mask)}\t" + "\t".join(values)
                )

        self._log("=" * 20)

    def solve(self):
        n = self.graph.size
        full_mask = (1 << n) - 1 # mask: all cities are visited
        memo = [[float("inf")] * n for _ in range(1 << n)] # all distances
are unknown
        parent = [[-1] * n for _ in range(1 << n)] # we don't what paths
are

        self._log("\n" + "=" * 20)
        self._log("HELD-KARP ALGORITHM")
        self._log(f"Number of cities: {n}")
        self._log(f"Full mask: {self._mask(full_mask)}")
        self._log("=" * 20)
        self._log("\nDistance matrix:")

        for row in self.graph.matrix:
            self._log("\t" + "\t".join(map(str, row)))

        self._log("\n\tInitializing base states")
        for u in range(n): # for each city
            # if all cities are visited, we only should come back to
start
            memo[full_mask][u] = self.graph.get_dist(u, 0)
            self._log(
                f"\tmemo[{self._mask(full_mask)}][{u}] = "
                f"distance({u}, 0) = {memo[full_mask][u]}"
            )

        self._log("\nMemo after initialization:")
        self._print_memo(memo)

```

```

self._log("\n\tFilling DP table")
for mask in range(full_mask - 1, 0, -1):
    # if 0-city (mask 00...001) isn't visited, it's not a valid
path
    # (all paths start from a 0-city)
    if not (mask & 1):
        continue

    self._log(
        f"\nProcessing mask {self._mask(mask)}, "
        f"visited cities: {self._visited(mask)}"
    )

    for u in range(n): # for each current city
        # if current city isn't visited, it's not a valid path
        # (how did we get here?..)
        if not (mask & (1 << u)):
            continue

        self._log(f"\tCurrent city: {u}")
        for v in range(n): # for each city to which we want to
travel
            # it's no use to visit cities we already have
visited
            # we visit each city only once
            if mask & (1 << v):
                continue

            # calculate the distance
            dist = self.graph.get_dist(u, v)
            # calculate overall distance of travelling
            # (using overall distance of path from v to the
finish)

            next_mask = mask | (1 << v)
            remaining = memo[next_mask][v]
            new_dist = dist + remaining

            self._log(
                f"\t\t{u} -> {v}: dist={dist}, "
                f"next mask={self._mask(next_mask)}, "
                f"remaining={remaining}, new_dist={new_dist}"
            )

            if new_dist < memo[mask][u]: # if it's optimal,
store it to memo
                memo[mask][u] = new_dist
                parent[mask][u] = v # (mask, u) -> v is
optimal

            self._log(
                f"\t\t\tUPDATE: memo[{self._mask(mask)}]
[{{u}}] "
                f"= {new_dist}, parent = {v}"
            )

        self._log(

```

```

        f"\tBest state value: memo[{self._mask(mask)}][{u}]"
    "
        f"= {memo[mask][u]}"
    )

    self._log("\nMemo after filling:")
    self._print_memo(memo)

    min_cost = memo[1][0] # we're in 0-city, and it's visited

    self._log("\n\tDynamic programming completed")
    self._log(f"Initial state: mask={self._mask(1)}, city=0")
    self._log(f"Minimum cost: {min_cost}")

    if min_cost == float("inf"):
        self._log("THERE IS NO PATH!")
        self._log("=" * 20 + "\n")

        return "no path", None

    # restoring the actual path
    self._log("\n\tRestoring the path")
    path = [0]
    mask = 1
    city = 0
    while mask != full_mask: # while we've not visited all the
cities
        next_city = parent[mask][city] # how did we get here?
        next_mask = mask | (1 << next_city)

        self._log(
            f"\tmask={self._mask(mask)}, city={city}, "
            f"parent={next_city}    ->    new
mask={self._mask(next_mask)}"
        )

        city = next_city
        path.append(city) # store it
        mask = next_mask # we've visited this city
        path.append(0) # come back to start

    self._log("Return to city 0")
    self._log(f"Optimal path: {' ' -> ' '.join(map(str, path))}")
    self._log("=" * 20 + "\n")
    return min_cost, path

def main():
    n = int(input())
    matrix = [list(map(int, input().split())) for _ in range(n)]
    debug = "--debug" in sys.argv[1:]
    solver = Alsh2
    #solver = HeldKarp
    cost, path = solver(Graph(matrix), debug).solve()

    if path is None:
        print("no path")
    else:

```

```
    print(cost)
    print(*path)

if __name__ == "__main__":
    main()
```